MyCodeSchool

Liste: exerciții în două stiluri

Partea 2 · David · exerciții

Fiecare problemă se rezolvă de două ori, în `liste.py`:
Stil A — ca în C++: numai cu `for`, `while`, `if`, indici `v[i]`, `len(v)`, `range(...)`, `append` și funcțiile scrise de tine. Fără `sum`, `max`, `min`, `sorted`, `sort`, `index`, `count`, `in`, `reverse`, fără felii (`v[1:3]`, `v[::-1]`) și fără liste-comprehension.
Stil B — stil Python: funcția cu același nume + `_py` și uneltele din teorie, secțiunea 4.
Ambele funcții întorc rezultatul cu `return`, nu citesc și nu afișează nimic, și **nu modifică lista primită**: unde rezultatul e o listă, o construiești nouă. Listele au cel puțin un element, iar numerele sunt întregi.

Înainte de problema 1 — proiectul tău

Repo nou: `python-liste`. În `liste.py` ai deja cele 22 de semnături, în ordinea fișei, cu corpul `pass` — înlocuiești `pass` cu codul problemei. Lista se citește de pe **o singură linie**, cu numerele despărțite prin spațiu. În `main.py` adaugi în `from liste import (...)` numele ambelor funcții ale problemei, iar codul din „În main()" îl pui în `def main() -> None:`.

liste.py

```
def suma_elementelor(v: list) -> int:
    ...


def suma_elementelor_py(v: list) -> int:
    ...
```

main.py

```
from liste import (
    suma_elementelor,
    suma_elementelor_py,
)


def main() -> None:
    v = [int(x) for x in input().split()]
    print(suma_elementelor(v), suma_elementelor_py(v))


if __name__ == "__main__":
    main()
```

La clasă

Aceiași algoritmi de vector, cu lista din Python.

1. Suma elementelor

În `liste.py`: `def suma_elementelor(v: list) -> int:` (stil A) și `def suma_elementelor_py(v: list) -> int:` (stil B)

Întoarce suma elementelor listei.

În main()

```
v = [int(x) for x in input().split()]
print(suma_elementelor(v), suma_elementelor_py(v))
```

Tastezi

```
7 3 10 5
```

Ieșire așteptată

```
25 25
```

2. Maximul și poziția lui

În `liste.py`: `def max_si_poz(v: list) -> tuple[int, int]:` (stil A) și `def max_si_poz_py(v: list) -> tuple[int, int]:` (stil B)

Întoarce, deodată, cel mai mare element și poziția lui. Dacă maximul apare de mai multe ori, întoarce poziția **primei** apariții.

În main()

```
v = [int(x) for x in input().split()]
m, p = max_si_poz(v)
print(m, p)
m, p = max_si_poz_py(v)
print(m, p)
```

Tastezi

```
4 9 2 9 1
```

Ieșire așteptată

```
9 1
9 1
```

3. Căutarea unei valori

În `liste.py`: `def cauta(v: list, x: int) -> int:` (stil A) și `def cauta_py(v: list, x: int) -> int:` (stil B)

Întoarce poziția primei apariții a lui x în listă, sau −1 dacă x nu apare deloc. Pozițiile se numără de la 0, ca în C++.

În main()

```
v = [int(x) for x in input().split()]
x = int(input())
print(cauta(v, x), cauta_py(v, x))
```

Tastezi

```
4 9 2 9 1
2
```

Ieșire așteptată

```
2 2
```

4. Lista inversată

În `liste.py`: `def inversata(v: list) -> list:` (stil A) și `def inversata_py(v: list) -> list:` (stil B)

Întoarce o **listă nouă**, cu elementele în ordine inversă. Lista primită rămâne neatinsă — de-aia al treilea `print` afișează lista de la început.

În main()

```
v = [int(x) for x in input().split()]
print(inversata(v))
print(inversata_py(v))
print(v)
```

Tastezi

```
1 2 3 4 5
```

Ieșire așteptată

```
[5, 4, 3, 2, 1]
[5, 4, 3, 2, 1]
[1, 2, 3, 4, 5]
```

De exersat

Probleme de bac, în ambele stiluri.

5. Doar elementele pare

În `liste.py`: `def doar_pare(v: list) -> list:` (stil A) și `def doar_pare_py(v: list) -> list:` (stil B)

Întoarce o listă nouă cu elementele pare, în ordinea în care apar.

În main()

```
v = [int(x) for x in input().split()]
print(doar_pare(v))
print(doar_pare_py(v))
```

Tastezi

```
7 2 9 4 6 3
```

Ieșire așteptată

```
[2, 4, 6]
[2, 4, 6]
```

6. Ștergerea unei valori

În `liste.py`: `def sterge_valoarea(v: list, x: int) -> list:` (stil A) și `def sterge_valoarea_py(v: list, x: int) -> list:` (stil B)

Întoarce o listă nouă, fără **niciuna** dintre aparițiile lui x. În C++ ștergeai mutând elementele la stânga; aici construiești lista rezultat.

În main()

```
v = [int(x) for x in input().split()]
x = int(input())
print(sterge_valoarea(v, x))
print(sterge_valoarea_py(v, x))
```

Tastezi

```
5 1 5 2 5 3
5
```

Ieșire așteptată

```
[1, 2, 3]
[1, 2, 3]
```

7. Elementul care apare cel mai des

În `liste.py`: `def cel_mai_frecvent(v: list) -> tuple[int, int]:` (stil A) și `def cel_mai_frecvent_py(v: list) -> tuple[int, int]:` (stil B)

Întoarce, deodată, elementul care apare cel mai des și de câte ori apare. Dacă mai multe elemente apar la fel de des, întoarce-l pe **cel mai mic**. În stilul B folosești `count`.

În main()

```
v = [int(x) for x in input().split()]
e, k = cel_mai_frecvent(v)
print(e, k)
e, k = cel_mai_frecvent_py(v)
print(e, k)
```

Tastezi

```
3 7 3 2 7 3
```

Ieșire așteptată

```
3 3
3 3
```

8. Interclasarea a două liste sortate

În `liste.py`: `def interclaseaza(a: list, b: list) -> list:` (stil A) și `def interclaseaza_py(a: list, b: list) -> list:` (stil B)

a și b sunt deja sortate crescător. Întoarce o listă sortată cu toate elementele lor. În stilul A mergi cu doi indici, ca în C++, fără să sortezi la final.

În main()

```
a = [int(x) for x in input().split()]
b = [int(x) for x in input().split()]
print(interclaseaza(a, b))
print(interclaseaza_py(a, b))
```

Tastezi

```
1 4 7 9
2 3 8
```

Ieșire așteptată

```
[1, 2, 3, 4, 7, 8, 9]
[1, 2, 3, 4, 7, 8, 9]
```

Provocări

Fără indicații în plus.

9. Lista sortată crescător

În `liste.py`: `def sortata(v: list) -> list:` (stil A) și `def sortata_py(v: list) -> list:` (stil B)

Întoarce o listă nouă, sortată crescător, fără să strici lista primită. În stilul A scrii sortarea ta, ca în C++.

În main()

```
v = [int(x) for x in input().split()]
print(sortata(v))
print(sortata_py(v))
print(v)
```

Tastezi

```
5 1 4 1 5 9 2
6
```

Ieșire așteptată

```
[1, 1, 2, 4, 5, 5, 6, 9]
[1, 1, 2, 4, 5, 5, 6, 9]
[5, 1, 4, 1, 5, 9, 2, 6]
```

10. Elementele distincte

În `liste.py`: `def distincte(v: list) -> list:` (stil A) și `def distincte_py(v: list) -> list:` (stil B)

Întoarce elementele care apar în listă, fiecare o singură dată, în ordinea primei apariții.

În main()

```
v = [int(x) for x in input().split()]
print(distincte(v))
print(distincte_py(v))
```

Tastezi

```
4 7 4 1 7 7 2
```

Ieșire așteptată

```
[4, 7, 1, 2]
[4, 7, 1, 2]
```

11. Rotirea cu k poziții la stânga

În `liste.py`: `def rotita(v: list, k: int) -> list:` (stil A) și `def rotita_py(v: list, k: int) -> list:` (stil B)

Întoarce o listă nouă în care fiecare element s-a mutat cu k poziții la stânga, iar cele care ies pe stânga reintră pe dreapta. k poate fi mai mare decât lungimea listei.

În main()

```
v = [int(x) for x in input().split()]
k = int(input())
print(rotita(v, k))
print(rotita_py(v, k))
```

Tastezi

```
1 2 3 4 5
2
```

Ieșire așteptată

```
[3, 4, 5, 1, 2]
[3, 4, 5, 1, 2]
```